

Tema 5: Integridad de mensajes, firmas y certificados

Docente : Héctor Xavier Limón Riaño
email: xavier120@hotmail.com

MAC

- ▶ El MAC protege la integridad del mensaje.
- ▶ Un atacante sin la llave no debe poder alterar el mensaje sin que se pueda detectar
- ▶ Mientras la llave se mantenga privada, el MAC provee autenticidad: el receptor sabe que sólo el emisor real pudo enviar el mensaje porque sólo así puede generarse un MAC correcto

HMAC

- ▶ Hash-based Message Authentication Code
- ▶ Es básicamente un hash keyed (no hay un término apropiado en español)
- ▶ Recordar que las funciones de hash son deterministas: dada la misma entrada, siempre se produce la misma salida
- ▶ Cuando un algoritmo es keyed, la salida no depende sólo del contenido de entrada, sino también de la llave

$$H(K \oplus opad, H((K \oplus ipad), text)), \text{ donde:}$$

- ▶ H es la función de hash
- ▶ K es la llave que debe cumplir ciertos requisitos
- ▶ $text$ es cualquier binario arbitrario que representa el contenido de entrada
- ▶ $opad$ e $ipad$ son dos valores estáticos que se presentarán más adelante

HMAC

- ▶ De hecho las funciones de hash también trabajan por bloque, por eso funciona la función de update, cada vez que se llama se aplica a cierto número de bloques
- ▶ Una función de hash opera un bloque a la vez y la salida de un bloque se encadena con el cálculo del siguiente bloque
- ▶ Por ejemplo, SHA-1 es de 64 bytes de bloque y SHA-256 es de 128 bytes de bloque

HMAC

En el RFC el tamaño de bloque de H se define como B , éstas son las reglas de restricción de K :

1. Si K es más corto que B , entonces a K se le tienen que agregar ceros de padding hasta ser de longitud B
2. Si K es más largo que B , entonces K se reduce aplicando H a K , osea $H(K)$
3. Si se aplicó 2 y la llave quedó muy corta, entonces se aplica 1

HMAC

- ▶ Si se aplican las reglas anteriores, siempre se va a tener una llave de longitud B
- ▶ En el RFC *ipad* se define como el byte 0x36 repetido B veces
- ▶ *opad* es el byte 0x5c repetido B veces
- ▶ Los pads parecen arbitrarios pero tienen un sentido de seguridad matemático que ayuda a mitigar problemas si los algoritmos de hash son vulnerados (como con MD5 o SHA-1)

HMAC

- ▶ Luego se computa $K \oplus opad$
- ▶ Para obtener el outer-hash final se concatena el resultado de arriba con el resultado del inner-hash y se aplica H
- ▶ El outer-hash es el HMAC de *text* keyed con K

Ejercicio

- ▶ Realiza una implementación manual de HMAC para SHA-256 de acuerdo a la explicación del RFC
- ▶ De acuerdo al tiempo que quede de clase el profesor podría implementar parte del ejercicio

HMAC

```
1 from cryptography.hazmat.backends import default_backend
2 from cryptography.hazmat.primitives import hashes, hmac
3
4 key = b"CorrectHorseBatteryStaple"
5 h = hmac.HMAC(key, hashes.SHA256(), backend=default_backend())
6 h.update(b"hello world")
7 h.finalize().hex() # HMAC en hexadecimal
```

CBC-MAC

- ▶ Similar a AES CBC
- ▶ Recordar que CBC es Cipher Block Chaining
- ▶ Un MAC a menudo es llamado un **tag** (etiqueta) del mensaje
- ▶ Es como una etiqueta que se agrega a un regalo: es información extra que se asocia al contenido principal

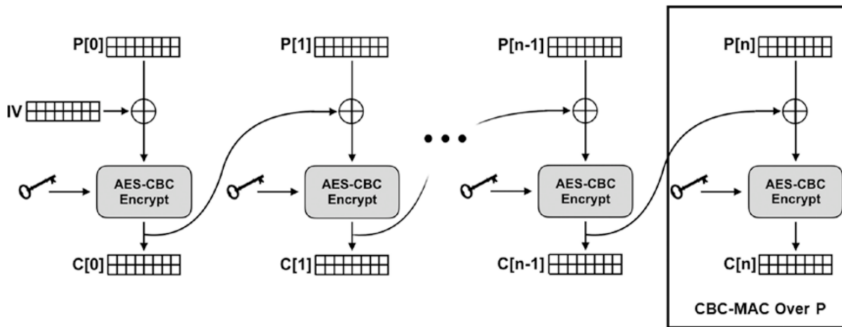
CBC-MAC

- ▶ En notación matemática un tag se denota a menudo como t
- ▶ Así, un MAC sobre un mensaje m_1 produce un tag t_1
- ▶ El par (m_1, t_1) es transmitido al receptor para ser verificado

CBC-MAC

- ▶ En el caso de AES, recuerda que se estaba limitado a cifrar 128 bits de bloque a la vez
- ▶ La idea de AES CBC es encadenar la salida de un bloque cifrado con el siguiente, para que no haya independencia entre bloques y sea más complejo alterar la integridad de los datos
- ▶ Si se alterara un bit en el primer bloque ese cambio crea una cascada de cambios en los bloques siguientes hasta llegar al último bloque
- ▶ Este efecto hace que el último bloque de CBC sea por si mismo un MAC

CBC-MAC



CBC-MAC

- ▶ En esencia CBC-MAC es AES CBC, recuperando el último bloque
- ▶ Pero se deben tomar en cuenta varias consideraciones de seguridad
- ▶ Veamos una implementación no segura

CBC-MAC: ataque antepuesto

- ▶ MAC prepend attack
- ▶ Este ataque funciona con la implementación insegura antes vista
- ▶ La idea básica es que si se tienen dos mensajes M_1 y M_2 y sus tags correspondientes t_1 y t_2 generados con la misma llave k , es posible generar un mensaje nuevo manipulado M_3 cuyo tag t_3 sea igual a t_1 o t_2
- ▶ Osea que es posible mandar un mensaje manipulado y hacer que parezca que es el mensaje M_1 o M_2

CBC-MAC: ataque antepuesto

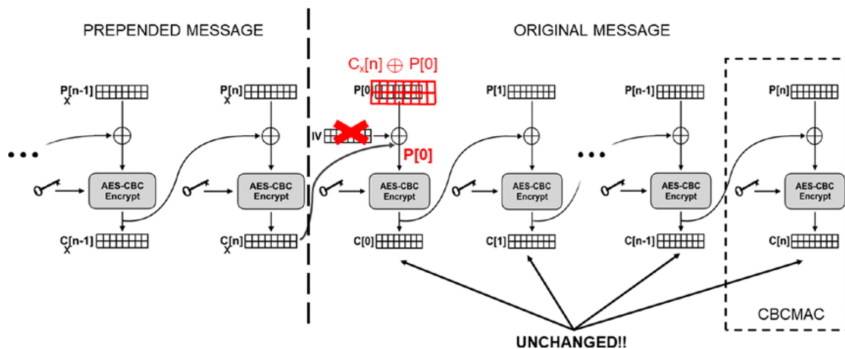
El proceso del ataque es el siguiente (anteponiendo M_1 a M_2):

- ▶ Recuperar el primer bloque de M_2 , denominado M_{2_1} (como es AES se toman los primeros 16 bytes)
- ▶ Recuperar el resto de bloques de M_2 , denominado M_{2_r}
- ▶ Obtener $M'_2 = t_1 \oplus M_{2_1}$
- ▶ $M_3 = M_1 + M'_2 + M_{2_r}$ (+ significa concatenación)
- ▶ Con esto $t_3 = t_2$

CBC-MAC: ataque antepuesto

- ▶ Básicamente lo que hace el ataque es remplazar el primer bloque de M_2 por uno manipulado
- ▶ La víctima verá todo el mensaje M_1 completo (posiblemente con su padding), un bloque que se verá posiblemente corrupto, y el mensaje M_2 sin su primer bloque

CBC-MAC: ataque antepuesto



CBC-MAC: ataque antepuesto

- ▶ Existen muchas soluciones a este ataque
- ▶ Una de ellas es por ejemplo agregar siempre la longitud del mensaje al inicio de éste, de esta manera puede revisarse si se agregó información extra
- ▶ Para hacer una implementación segura se requiere además:
 - ▶ Si además los datos se están cifrando con AES-CBC, no se debe usar la misma llave para cifrar y obtener el MAC
 - ▶ El IV debe ser diferente de cero
- ▶ Lo mejor es utilizar una versión segura estable, por ejemplo CMAC de la biblioteca cryptography que es una versión mejorada de CBC-MAC descrita en el RFC 4493

CBC-MAC

```
1 from cryptography.hazmat.backends import default_backend
2 from cryptography.hazmat.primitives import cmac
3 from cryptography.hazmat.primitives.ciphers import algorithms
4
5 c = cmac.CMAC(algorithms.AES(key), backend=default_backend())
6 c.update(b"message to authenticate")
7 mac = c.finalize()
```

HMAC y CBC-MAC

- ▶ Tanto HMAC como CBC-MAC son buenas opciones para un algoritmo MAC
- ▶ HMAC puede ser más eficiente en la mayoría de sistemas, con excepción de aquellos que tienen hardware especializado para AES
- ▶ Ante la duda, utiliza HMAC

Cifrado y MAC

- ▶ En muchas circunstancias un mensaje necesita ser cifrado y protegido contra modificaciones
- ▶ Si lo analizan, en lo que se vio anteriormente no se especifica si los mensajes para los cuales se derivaba el MAC estaban cifrados o no, en realidad podría ser cualquiera de las dos, los métodos funcionarían igual
- ▶ Pero, si se va a cifrar ¿En qué momento hay que derivar el MAC? ¿Antes o después de cifrar?

Cifrado y MAC

Existen tres aproximaciones:

- ▶ Cifra luego aplica MAC (Encrypt-Then-MAC): el MAC se aplica al texto cifrado
- ▶ Aplica MAC luego cifra (MAC-Then-Encrypt) (usado por versiones iniciales de TLS): se aplica el MAC al texto plano, luego se cifra el texto plano junto con el MAC
- ▶ Cifra y aplica MAC (Encrypt-And-MAC) (usado por SSH): se aplica el MAC sobre el texto plano, pero no se cifra el MAC, sólo el texto plano

Cifrado y MAC

- ▶ Lo que recomienda la mayoría de los criptógrafos (aunque no todos), es utilizar cifra luego aplica MAC (Encrypt-Then-MAC)
- ▶ Algunas vulnerabilidades han sido encontradas para aplica MAC luego cifra (MAC-Then-Encrypt), de hecho por eso ya no se usa en TLS. Por ejemplo el ataque de padding oráculo a CBC sólo funciona con este esquema
- ▶ Cifra luego aplica MAC parece contra-intuitivo, puesto que lo que queremos saber es si los datos originales (texto plano) llegaron bien

Introducción

- ▶ La autenticidad es al menos tan importante como la confidencialidad
- ▶ Si las claves se comprometen es posible que pase un tiempo para que las entidades se den cuenta, además, el proceso de revocación puede ser tardado
- ▶ Las firmas digitales mitigan este problema mediante criptografía asimétrica

Firmas digitales

Siguen dos pasas básicos:

- ▶ Calcular el hash del mensaje
- ▶ Cifrar el hash mediante la llave privada

Firmas digitales

- El hash cifrado es la firma digital del emisor
- La firma es transmitida junto con el mensaje (cifrado o no)
- El receptor calcula el hash del mensaje, descifra la firma digital mediante la llave pública y comprueba que los dos hashes coincidan

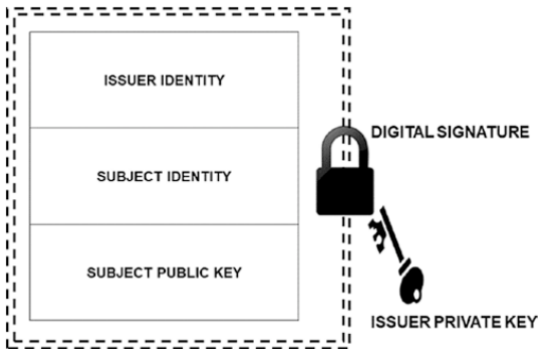
Cifrar y firmar

- ▶ Si tienes que utilizar firmas digitales para un mensaje, lo correcto es firmar luego cifrar, esto es, **se firma el texto plano y luego se cifra tanto el mensaje como la firma**
- ▶ Aunque no existen ataques conocidos para este esquema, los más paranoicos pueden utilizar un esquema: firma luego cifra luego vuelve a firmar
- ▶ Lo anterior te da una firma para la información original y otra para los datos cifrados, con doble autenticación
- ▶ Existen otras técnicas conocidas como singcryption que no se verán en el curso

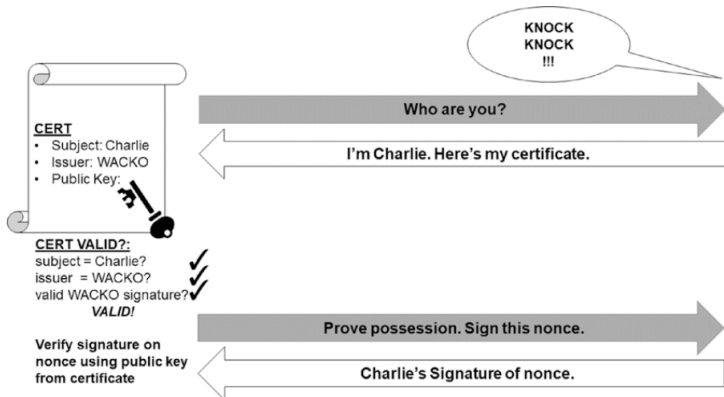
Curvas elípticas

```
1 from cryptography.hazmat.backends import default_backend
2 from cryptography.hazmat.primitives import hashes
3 from cryptography.hazmat.primitives.asymmetric import ec
4
5 private_key = ec.generate_private_key(
6     ec.SECP384R1(), default_backend())
7 public_key = private_key.public_key()
8
9 data = b"this is some data I'd like to sign"
10 signature = private_key.sign(data,
11     ec.ECDSA(hashes.SHA256()))
12
13 public_key.verify(signature, data, ec.ECDSA(hashes.SHA256()))
```


Certificados



Certificados



Revocación de certificados

- ▶ Si la llave privada de un certificado generado por un CA se compromete, el CA debe determinar una forma de hacer saber a quienes tienen el certificado ya no es confiable
- ▶ Esto no es tan sencillo
- ▶ Una de las razones para dejar de utilizar un registro global de llaves públicas y usar certificados era poder hacer verificaciones offline
- ▶ ¿Cómo puede un sistema de verificación offline tener información de revocación?

Revocación de certificados

- ▶ La respuesta corta a la pregunta anterior es: no se puede
- ▶ Existen dos opciones:
 - ▶ El proceso de verificación debe tener un componente en tiempo real (online)
 - ▶ La revocación no puede ser actualizada en tiempo real

100%

- ▶ Aunque el enemigo no pueda alterar el mensaje ni leerlo, puede reenviarlo
- ▶ Esto crea confusión y puede hacer que el enemigo adelante una estrategia

Prevención

- ▶ Algunos sistemas usan un timestamp en vez de nonces aleatorios
- ▶ Con un timestamp el receptor puede rechazar un mensaje que sea demasiado viejo
- ▶ El problema de esta aproximación es que las partes en comunicación deben mantener sus relojes sincronizados, lo cual es más complejo de lo que parece
- ▶ Dado que no es posible tener una sincronización exacta, debido a retraso introducido por diferentes fuentes de latencia (velocidad de red, hops, etc.) el receptor debe aceptar mensajes dentro de una ventana de tiempo para lidiar con el error de tiempo
- ▶ ¿De qué tamaño debe ser la ventana de tiempo? otra vez el atacante puede encontrar patrones explotables

